

BondNest

System Documentation

A social networking web application with posts,
moderation, messaging, and notifications

Technical record of the implemented system
September 2026

Author
Tolentino, Lawrence Dave P.

Live application: <https://bondnest.up.railway.app/>
GitHub repository: <https://github.com/rproject1324/BondNest>

Note: This is a second-year level, unfinished project. Known limitations are documented for transparency.

Table of Contents

1. Project Overview
2. Objectives
3. System Features and Functionalities
4. Technology Stack
5. System Design and Architecture
6. Major Workflows
7. Database Design
8. User Interface and Page Documentation
9. Authentication, Authorization, and Account Security
10. External Services: Roles and Setup
11. Environment Variables and Configuration
12. Local Development Setup
13. Railway Deployment
14. Testing and Current Behavior
15. Known Limitations and Current System Status
16. Lessons Learned
17. Future Improvements
18. Project Links

19. Acknowledgements

1. Project Overview

BondNest is an account-based social networking web application built with vanilla PHP and PostgreSQL. Registered users create text/image posts, like and comment, exchange direct messages, and receive moderation notifications. Administrators review content through a dashboard that supports approving, placing on hold, warning, and deleting posts.

Important context. BondNest was developed as a second-year level, unfinished project. It is not feature-complete: several flows are partial, debug/test artifacts remain in the repository, and some pages contain dead links or stub scripts. This document records the system as implemented, including its gaps, so future maintainers and reviewers see an accurate picture rather than an idealized one.

Item	Current value
Project name	BondNest
Project type	Full-stack web application (server-rendered PHP, no framework)
Intended users	Students / general users (campus social network)
Primary live host	Railway (PHP built-in server + PostgreSQL)
Current status	Deployed but unfinished; known limitations documented later

2. Objectives

2.1 General objective

To provide a small social platform where users can publish posts, interact through likes, comments and messages, and where administrators can moderate content through approval, hold, warning, and deletion workflows with user-facing notifications.

2.2 Specific objectives

- Support registration with email OTP verification, session login, and password recovery.
- Provide post creation with images, likes, threaded comments/replies, and personal profiles.
- Provide 1:1 direct messaging with unread tracking and user search.
- Give administrators a dashboard with statistics, post search, and moderation actions.
- Notify users of moderation outcomes (approved, on hold, warned, deleted) with detail views.
- Enforce a shared password-strength policy on signup, reset, and password change.
- Deploy the stack on Railway with PostgreSQL and persistent uploads.

3. System Features and Functionalities

3.1 Guests (unauthenticated)

- Register with username, email, names, gender, birthday, and a policy-checked password; must accept Terms & Privacy.
- Verify the account with a six-digit email OTP (Brevo) on the verification page.
- Sign in with username or email plus password; recover a forgotten password via email OTP.

3.2 Authenticated users

- Homepage feed of approved/posted posts with create/edit/delete of own posts and image upload.
- Likes with toggle state and counts; comments with replies, edit, and delete.
- Profile pages (own + others) with photo, bio, details, and per-user post lists.
- Settings: personal info, profile photo, bio, email change via OTP, password change.
- Direct messaging inbox, conversation view, user search, read markers, unread badges.
- Notifications bell (5 per page, paginated): post approved / on hold / warning / deleted, each linking to a detail inbox with Clear-Read support.
- Online presence (Active now / Away / Last seen) derived from activity heartbeats.

3.3 Administrator

A user whose email matches the configured admin email (or whose `is_admin` flag is set) opens `admin.php`: statistics cards, post table with search/filter, and approve / hold / delete (with reason) / warn (with reason) actions. Deletions and warnings archive a snapshot of the post for later detail views. Admin is a configured operator role, not a public self-serve role.

3.4 Cross-cutting behavior

- PHP sessions gate every authenticated page; guests are redirected to the login page.
- Post age labels ("N seconds ago") are server-rendered, then kept live by a server-clock-synced client timer.
- Uploads persist under `UPLOADS_DIR` (intended for a Railway volume).
- Toast notifications confirm actions across homepage, profile, and admin flows.

4. Technology Stack

Category	Technology / service
Frontend	Server-rendered PHP templates, custom CSS, vanilla JavaScript; Bootstrap Icons 1.11, Font Awesome 6
Backend	Vanilla PHP 8.1+ (deployed on PHP 8.3) + PDO; no framework; idempotent <code>migrate.php</code> bootstrap
Database	PostgreSQL on Railway (psycopg2-style via <code>pdo_pgsql</code>); MySQL locally under XAMPP
Authentication	PHP sessions, <code>password_hash()</code> / <code>password_verify()</code> , six-digit email OTPs
Email / OTP	Brevo transactional HTTP API (<code>api.brevo.com/v3/smtp/email</code>)
Version control	Git / GitHub
Hosting	Railway (web + Postgres + volume); PHP built-in server via Procfile

5. System Design and Architecture

5.1 Runtime architecture

The browser interacts with PHP pages that authenticate the session, read/write PostgreSQL through PDO, store images on a volume-backed uploads directory, and send mail through Brevo. Admin moderation writes both the post row and a notification row (plus snapshot rows for warnings/deletions). A small set of JSON endpoints serves likes, comments, messages, notification polling, and admin AJAX.

BondNest — Runtime Architecture

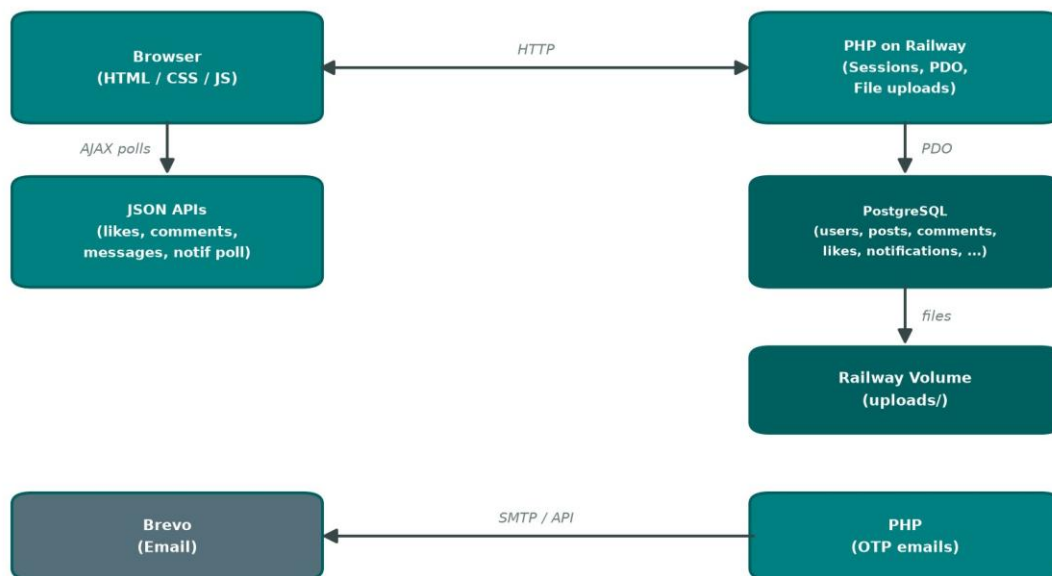


Figure 1. Runtime architecture of BondNest showing browser, PHP server, PostgreSQL, uploads, and Brevo email service.

5.2 Why this shape

The project prioritizes a small, readable codebase over framework ceremony: one PHP file per page, one endpoint per action, and a single migration script that creates/patches tables on boot. The trade-off is duplication (notably the profile page variants) and the rough edges listed in chapter 15.

6. Major Workflows

6.1 Registration and email OTP

BondNest — Registration and Email OTP

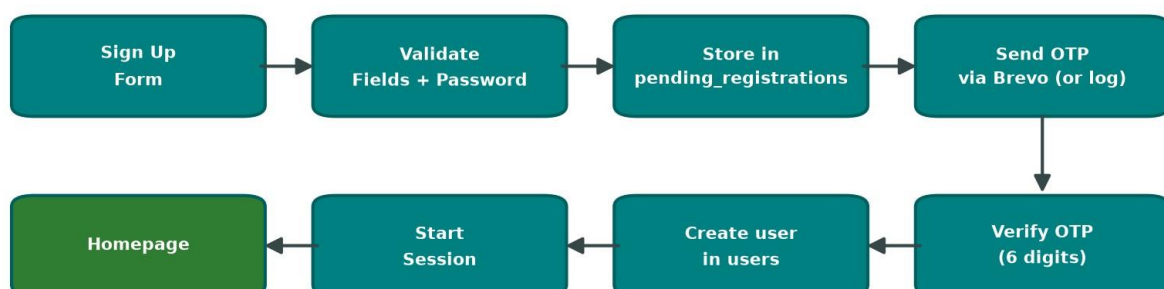


Figure 2. Registration and email OTP verification workflow.

Signup validates fields and the password policy, stores a hashed password in `pending_registrations` with a six-digit OTP expiring in 10 minutes, and mails the code via Brevo (dev-mode logs it when keys are absent). Verification creates the users row, starts a session, and routes to the homepage (or admin).

6.2 Login and password recovery

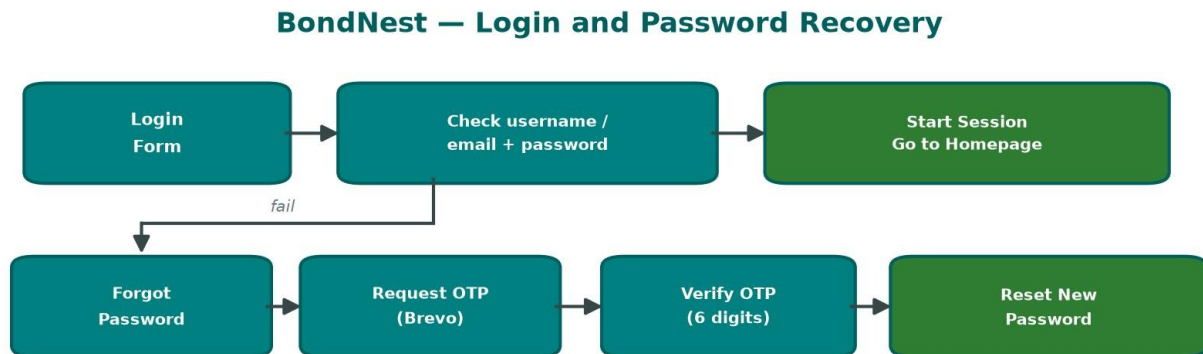


Figure 3. Login and password recovery workflow.

Login accepts username or email (case-insensitive) plus password with a generic failure message. Forgotten passwords use a three-step Brevo OTP flow (request → verify → reset under the password policy).

6.3 Posting and moderation

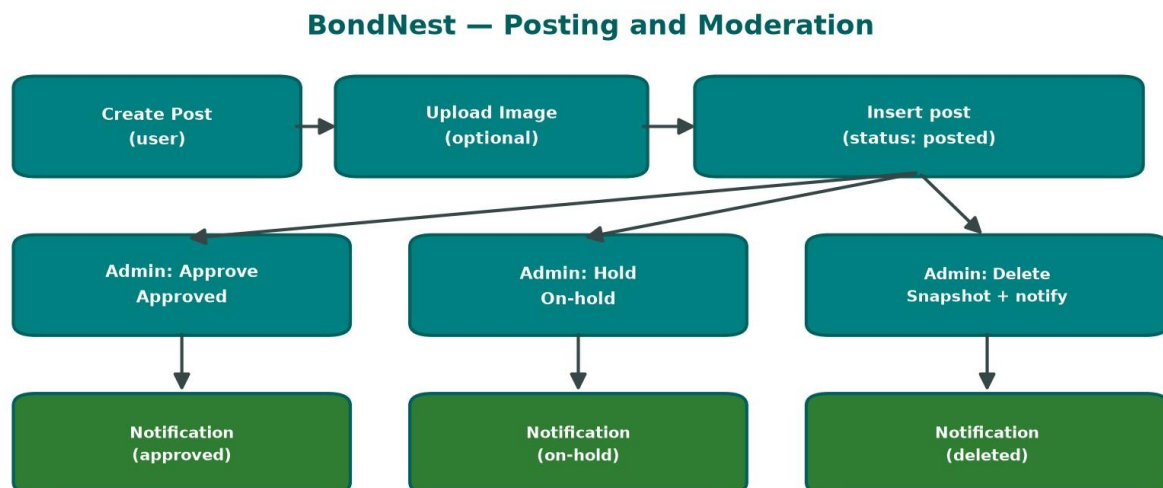


Figure 4. Posting and moderation workflow.

Users publish text/image posts (status posted). Admins approve (→ approved), hold (→ on-hold), warn (snapshot into warnings + notification), or delete (snapshot into `deleted_posts` + notification, then row removal). Approve/hold

notifications additionally embed a post snapshot so detail views render even if the post row later disappears. Live status polling updates badges without refresh.

6.4 Messaging and notifications

BondNest — Messaging and Notifications

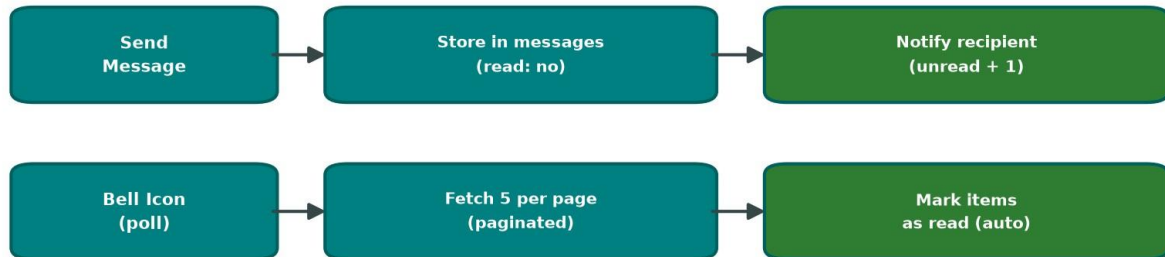


Figure 5. Messaging and notification polling workflow.

Direct messages store sender/receiver/content with read flags and soft delete. Moderation outcomes create typed notifications; the navbar bell fetches 5 per page, marks displayed items read, and deep-links each type to its inbox (warnings / held / approved / deleted).

7. Database Design

Production uses PostgreSQL via DATABASE_URL; local development defaults to MySQL on XAMPP. `migrate.php:runMigration()` creates tables if missing and patches columns on boot. Time is stored in UTC.

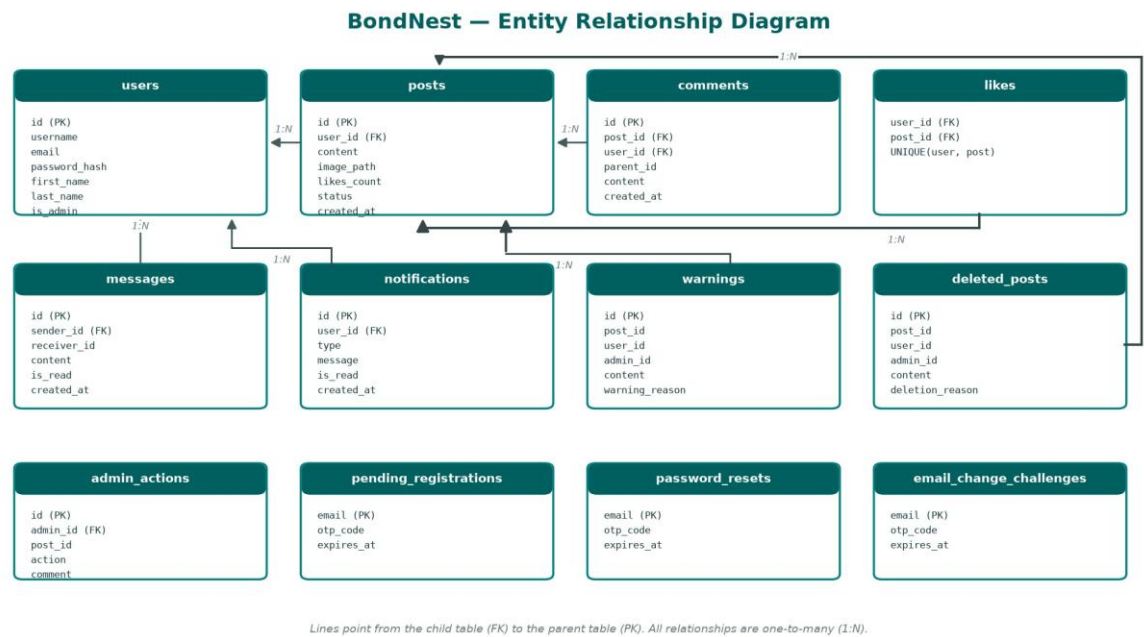


Figure 6. Entity-Relationship Diagram (simplified) showing core tables and foreign key relationships.

Table	Role
users	Identity, password hash, profile fields, last_activity/user_status, is_admin
posts	Content, image_path, likes counter, created/updated_at, moderation status
comments	Post comments with parent_id for replies
likes	Dedup join UNIQUE(user_id, post_id)
notifications	Typed inbox (post_warning/deleted/on_hold/approved), message, reference_id, is_read, additional_data
messages	1:1 messages with image_path, read flags, soft delete
warnings	Snapshot archive of warned posts + warning_reason
deleted_posts	Snapshot archive of deleted posts + deletion_reason
admin_actions	Audit log of admin actions with comments
pending_registrations	Unverified signups keyed by email with OTP + expiry
password_resets	Forgot-password OTPs keyed by email
email_change_challenges	Logged-in email-change OTP challenges

8. User Interface and Page Documentation

This section documents each user-facing page, including its purpose, layout, user actions, and server interactions. Screenshot placeholders are included for each page.

8.1 Sign In (index.php)

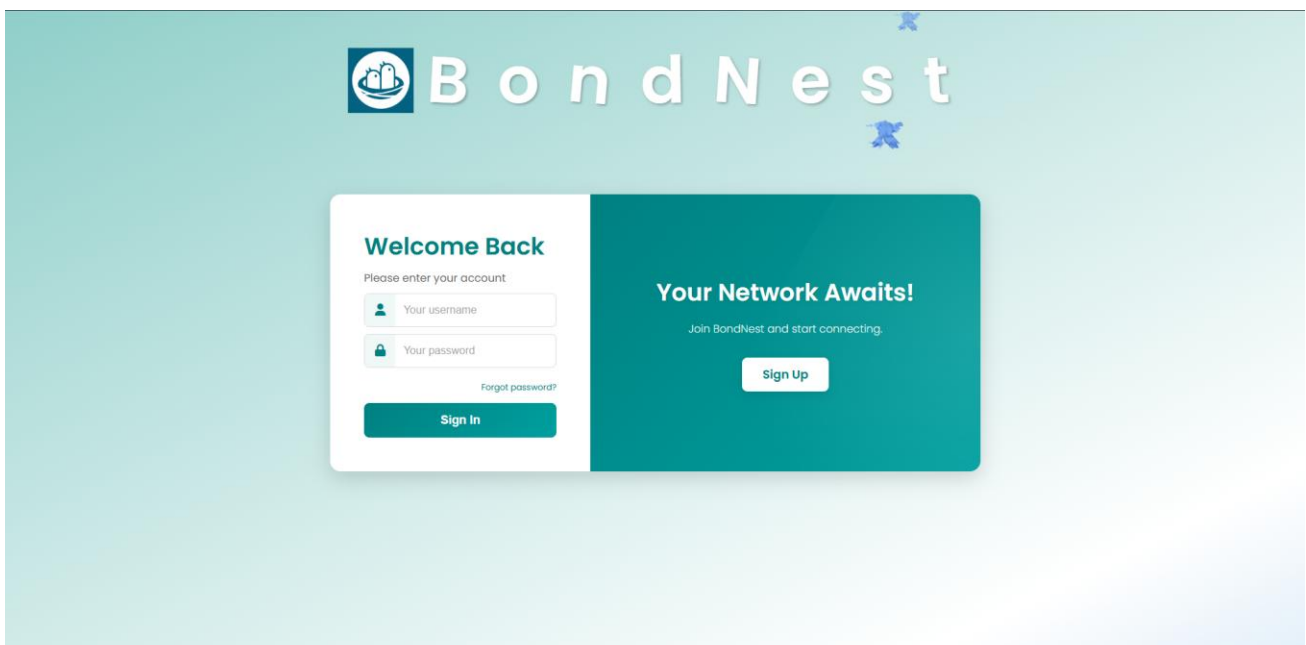
Purpose. Provides login for existing users and a three-step password recovery flow via email OTP.

Access. Public. Redirects to homepage if already signed in.

Main UI. Centered login card with username/email and password fields. A "Forgot Password?" link triggers an in-page modal sequence: (1) enter email to request an OTP, (2) enter the 6-digit code, (3) set a new password. Status messages confirm success or report errors inline.

Actions. Submit login credentials; request password-reset OTP; verify OTP code; submit new password. All actions use standard form POST with page reload.

Server interaction. POST handler queries the users table for credential matching. `bondGenerateOtp()` creates a 6-digit code; `bondSendPasswordResetOtp()` sends it via Brevo API (dev-mode logs it when keys are absent). Reset codes stored in `password_resets` with 10-minute expiry. Password update uses `password_hash()` / `password_verify()`.



8.2 Sign Up (signup.php)

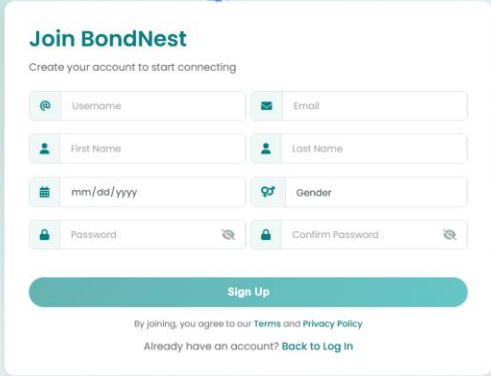
Purpose. Registers new users with identity fields, live password-strength feedback, and Terms & Privacy consent.

Access. Public. Redirects to homepage if already signed in.

Main UI. Two-column form: left column for username, email, first name, last name, birthday, gender; right column for password and confirm password with a live strength meter (bar + label). A shared policy message below the password fields lists the requirements (12–64 chars, upper/lower/digit/special, no spaces, not common, no username/email/name substrings). The "I agree" checkbox opens a Terms & Privacy modal.

Actions. Inline validation on blur for each field; username availability check via AJAX; agree-and-continue validates the entire form before submission; success navigates to the verification page.

Server interaction. AJAX endpoint checks username availability against the users table. POST handler validates all fields server-side using the same password policy, generates a 6-digit OTP via `generateOtp()`, stores the pending registration in `pending_registrations` with a 10-minute expiry, and sends the OTP via Brevo. Redirects to `verify-registration.php`.



The registration form is titled "Join BondNest" and includes the subtext "Create your account to start connecting". It features several input fields: Username, Email, First Name, Last Name, a date field (mm/dd/yyyy), Gender, Password, and Confirm Password. Each field has a corresponding icon (e.g., @ for email, person for name, calendar for date, gender symbol for gender, and lock for password). A teal "Sign Up" button is positioned below the fields. At the bottom, there is a disclaimer: "By joining, you agree to our [Terms](#) and [Privacy Policy](#)" and a link: "Already have an account? [Back to Log In](#)".

8.3 Verify Registration (verify-registration.php)

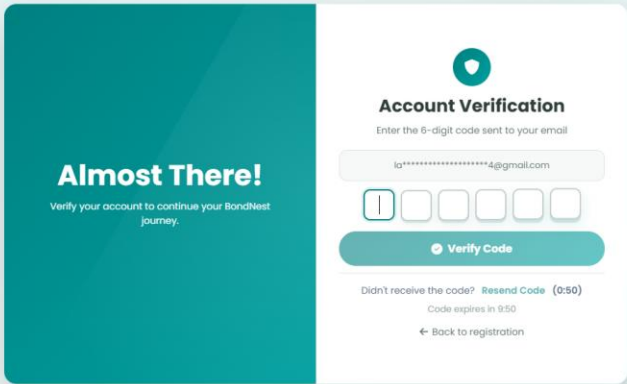
Purpose. Completes registration by verifying the user's email address with a 6-digit OTP.

Access. Public (requires a pending registration session). Redirects to homepage if no pending registration.

Main UI. Single OTP input with six digit fields. A "Resend Code" link generates a new OTP. Status messages confirm verification or report an invalid/expired code.

Actions. Enter 6-digit code and submit; resend OTP; after successful verification, user is signed in.

Server interaction. POST handler validates the OTP against the stored code in pending_registrations. On success, creates the users row, starts a session, and redirects to the homepage (or admin if the email matches the admin allow-list). Resend generates a new OTP and resets the expiry.



The verification screen is split into two panels. The left panel is teal and says "Almost There!" with the subtext "Verify your account to continue your BondNest journey." The right panel is white and titled "Account Verification" with a shield icon. It prompts the user to "Enter the 6-digit code sent to your email". Below this is an email field containing "lg*****4@gmail.com". There are six input boxes for the 6-digit code, with the first box containing the digit "1". A teal "Verify Code" button is below the input boxes. At the bottom, there is a link "Didn't receive the code? [Resend Code](#) (0:50)" and a note "Code expires in 0:50". A "Back to registration" link with a left arrow is at the very bottom.

8.4 Homepage (homepage.php)

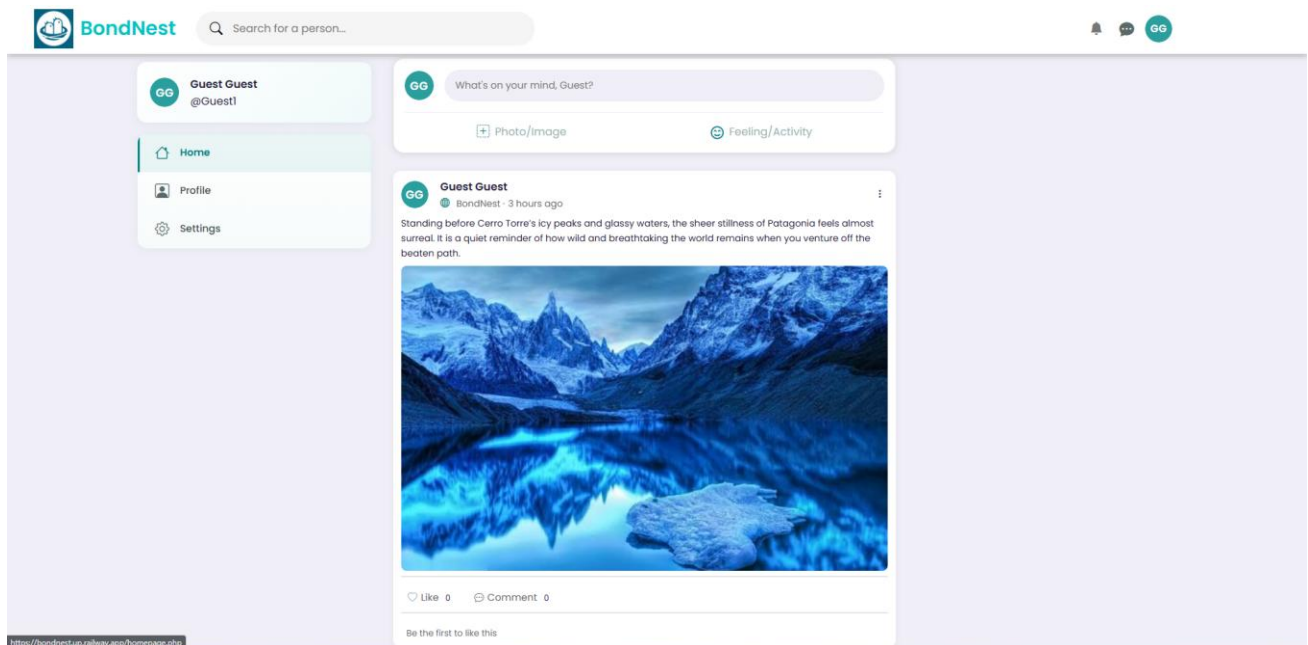
Purpose. Displays the main social feed with post creation, likes, comments, and real-time age labels.

Access. Requires login. Redirects to login page if not authenticated.

Main UI. Top section: create-post box with text area, image upload button, and "Post" button. Below: scrollable feed of approved/posted posts, each showing author avatar (initials fallback), post content, image (if any), like button with count, comment count, and a "..." menu for edit/delete on own posts. Toasts confirm actions. Age labels ("N seconds ago") are kept live via server-clock-synced client timer.

Actions. Create post (text + optional image); edit own post (modal); delete own post (confirmation); toggle like (AJAX); add comment; reply to comment; edit/delete own comments.

Server interaction. Posts fetched from posts table where status is approved or posted, joined with users for author info. AJAX endpoints: `create_post.php`, `update_post.php`, `delete_post.php`, `like_post.php`, `add_comment.php`, `update_comment.php`, `delete_comment.php`, `get_posts_status.php`. Server-time endpoint provides clock sync.



8.5 Profile (profile-page.php)

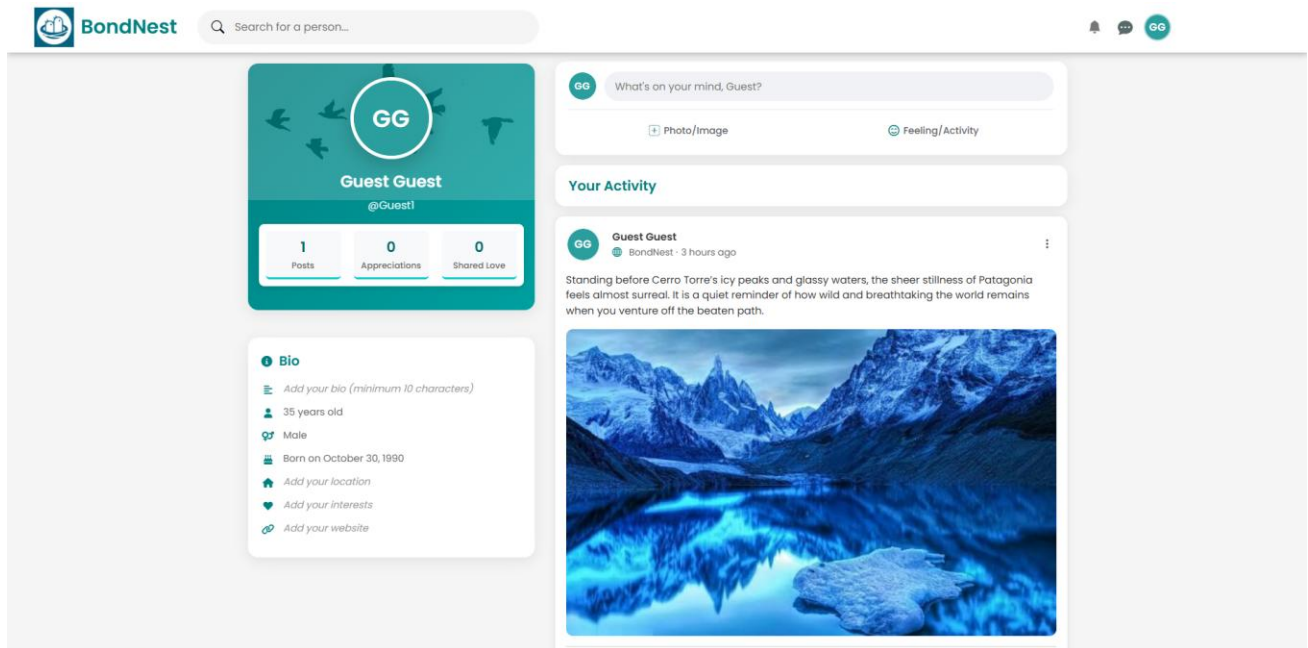
Purpose. Displays user profiles with bio, details, and per-user post activity.

Access. Requires login. Shows own profile or other users' profiles.

Main UI. Cover card with profile picture (or initials fallback), display name, username, and bio. Details section shows gender, birthday, and other personal info. Below: tabbed view of the user's posts with moderation badges (approved, on-hold, etc.). Edit modal allows updating profile picture, bio, and details.

Actions. View own or others' profiles; edit profile (picture, bio, details); manage own posts (edit/delete from profile context).

Server interaction. Fetches user data from users table and posts from posts table. AJAX username availability check for username changes. Profile update POSTs to update the users row. Profile picture uploaded to uploads/ directory.



8.6 Settings (settings.php)

Purpose. Manages personal info, profile photo, bio, email change (via OTP), and password change.

Access. Requires login.

Main UI. Tabbed interface with sections: Personal Info (first/last name, username, birthday, gender), Profile Photo (upload with preview), Bio, Email Change (enter new email → verify OTP → confirm), Password Change (current + new + confirm with strength meter). Online presence settings visible.

Actions. Update personal info; upload/change profile photo; change email (OTP flow); change password (with policy validation); toggle presence settings.

Server interaction. Runs migrate.php on load. OTP generation and Brevo email for email change. Password change validates current password with password_verify(), then updates with password_hash(). Profile photo uploaded to uploads/ directory. All updates write to the users table.

The screenshot shows the BondNest user interface. At the top, there's a header with the BondNest logo, a search bar, and user avatars. A left sidebar contains navigation links: Home, Profile, and Settings (which is highlighted). The main content area is titled 'Settings' with the subtitle 'Manage your personal information and security preferences'. Below this are two tabs: 'Personal Information' (selected) and 'Security'. The 'Profile Details' section includes a profile card for 'Guest Guest' with a 'Change Photo' button. Below the card are form fields for 'First Name' (Guest), 'Last Name' (Guest), 'Username' (Guest), 'Email Address' (rproject.1324@gmail.com), 'Gender' (Male), 'Birthday' (10/30/1990), 'Age', and 'Location'.

This screenshot shows the 'Security' tab in the BondNest settings. It features a green notification box stating 'Your account is protected. Use a strong, unique password to maintain account security.' Below this are input fields for 'Current Password', 'New Password', and 'Confirm New Password'. A 'Password Requirements' section lists several criteria: 'At least 8 characters', '1 lowercase letter', '1 uppercase letter', '1 number', '1 special character', and 'Passwords match'. An 'Update Password' button is located at the bottom right of the form.

8.7 Messages (message.php)

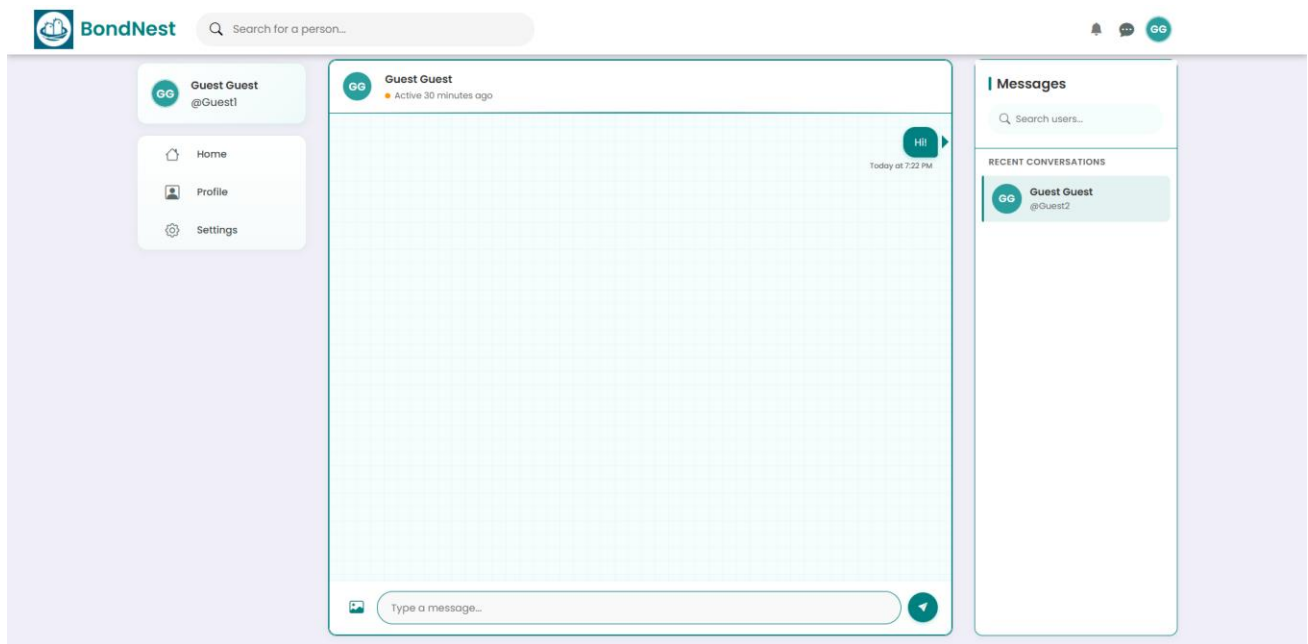
Purpose. Provides 1:1 direct messaging with inbox, conversation view, and user search.

Access. Requires login.

Main UI. Two-panel layout: left panel shows conversation list with unread badges and user avatars (initials fallback); right panel shows the selected conversation thread with message bubbles, timestamps, and a text input with send button. Top bar includes a user search field for finding new contacts.

Actions. Search users (AJAX); open conversation; send message; view message history; mark messages as read; soft-delete own messages.

Server interaction. User search queries users table (excludes admin users). Messages fetched from messages table with sender/receiver join. Send POST inserts into messages with `is_read = false`. Read status updated on conversation open. Soft delete sets `sender_del` or `receiver_del` flag.



8.8 Notification Inboxes

8.8.1 Approved Posts (approved_posts.php)

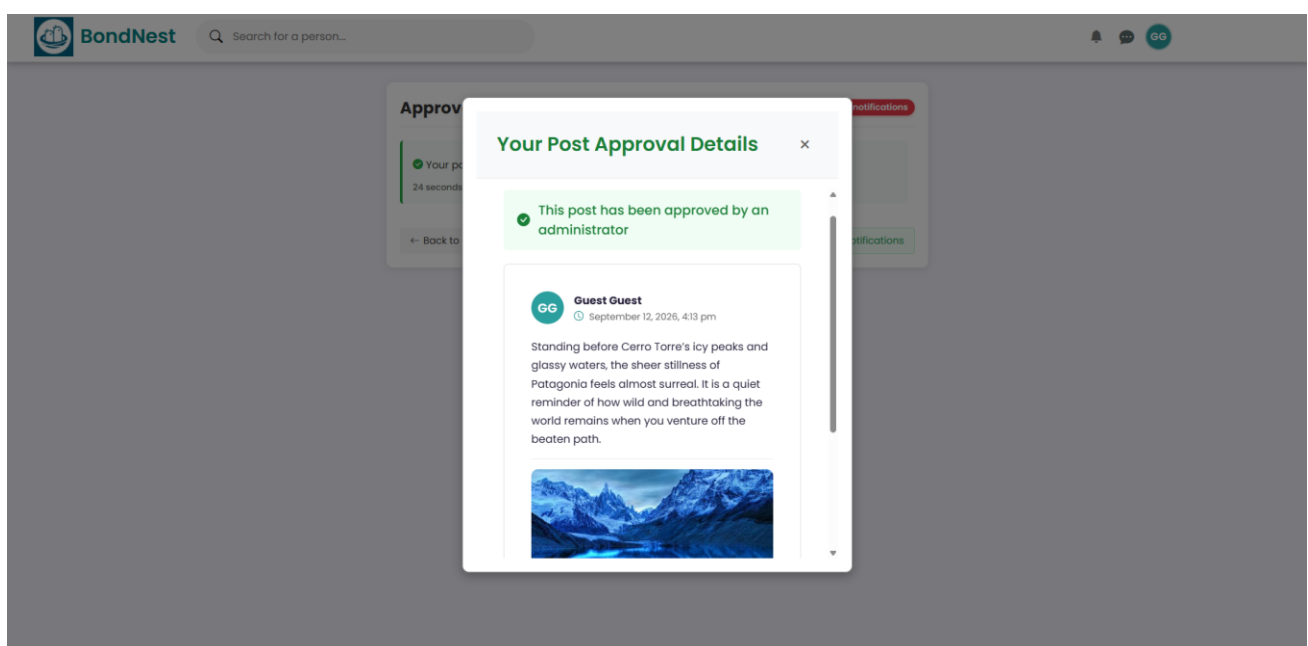
Purpose. Shows notifications when a held post gets approved by an admin.

Access. Requires login (scoped to own notifications only).

Main UI. List of post_approved notification cards, each showing the post title/content, approval timestamp, and a "View Details" button. Mark-read and Clear-All buttons at the top.

Actions. Mark individual notification as read; clear all read notifications; view post details in a modal.

Server interaction. Queries notifications table where type = 'post_approved'. Post snapshot embedded in notification for detail view. mark_read and clear_all action handlers.



8.8.2 Held Posts (held_posts.php)

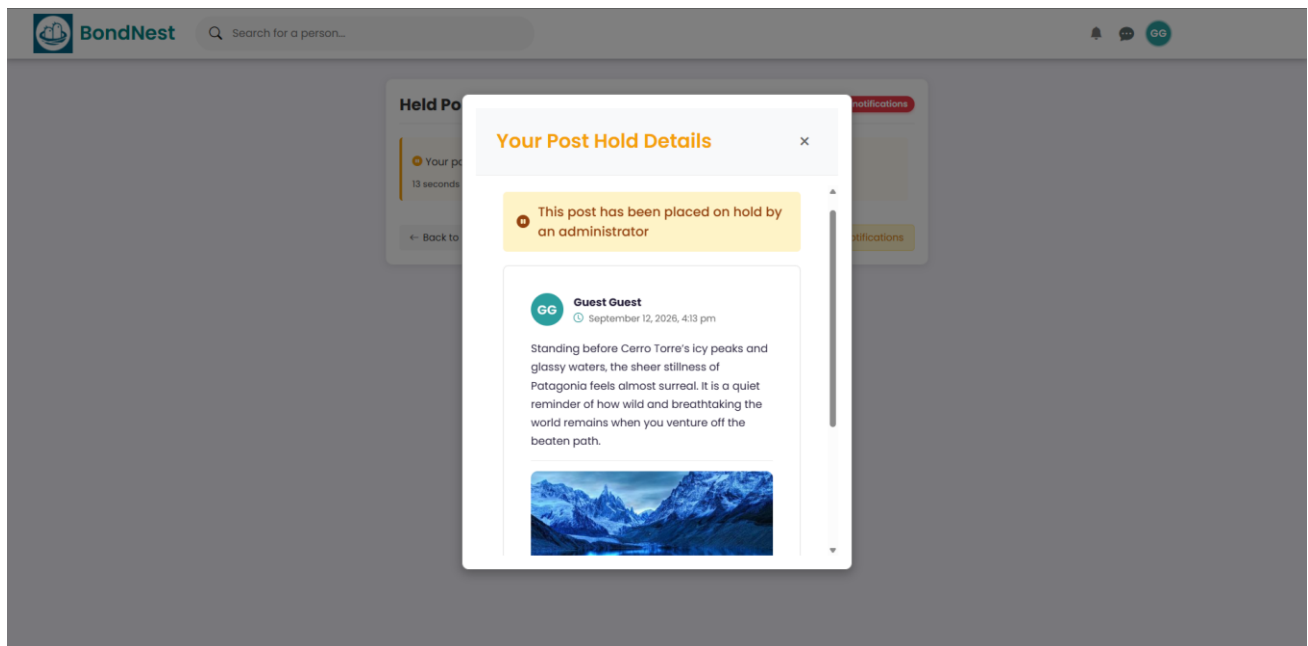
Purpose. Shows notifications when a post is placed on hold for review.

Access. Requires login (scoped to own notifications only).

Main UI. List of post_on_hold notification cards with post snapshot preview, hold reason, and timestamps. Mark-read and Clear-All buttons.

Actions. Mark notification as read; clear all read notifications; view held post details.

Server interaction. Queries notifications table where type = 'post_on_hold'. Post snapshot embedded in notification. mark_read and clear_all handlers.



8.8.3 Warnings (warnings.php)

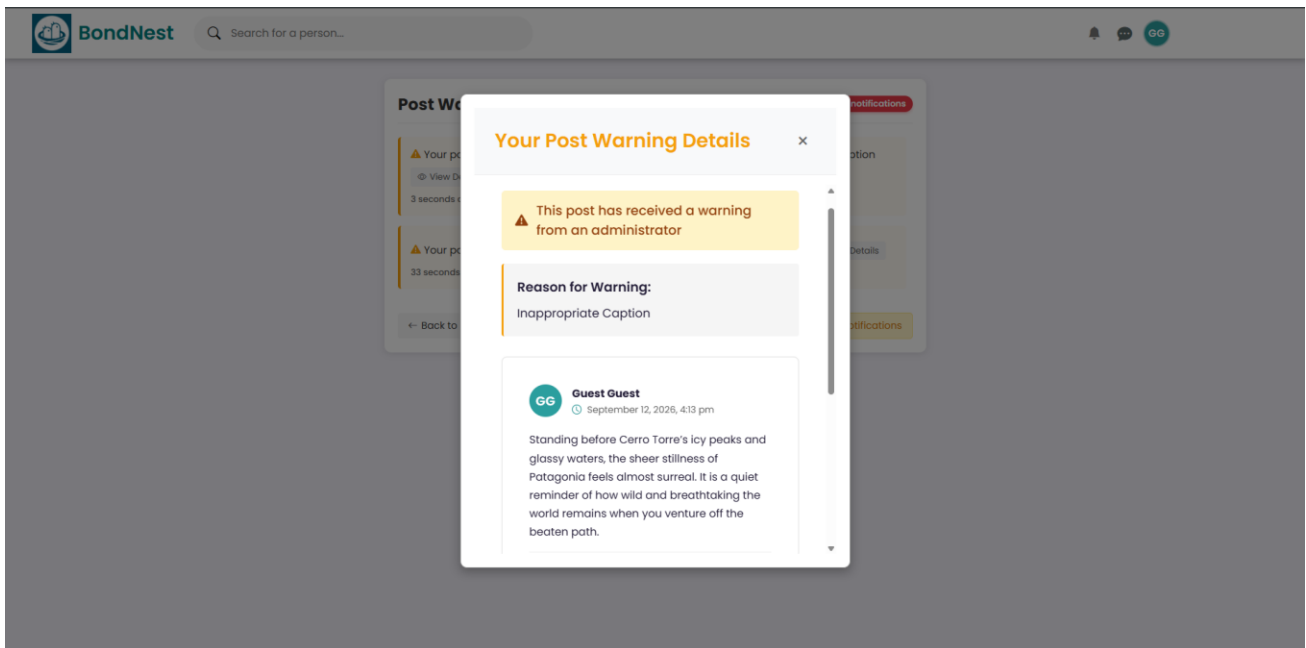
Purpose. Displays warnings issued by admins with post snapshot context.

Access. Requires login (scoped to own notifications only).

Main UI. List of warning notification cards showing the warned post content, warning reason, admin name, and timestamps. The warnings table is created on first run if missing.

Actions. Mark notification as read; clear all read warnings.

Server interaction. Creates warnings table if not exists (PostgreSQL/MySQL variants). Queries notifications table where type = 'warning'. Snapshot table preserves: original_post_id, user_id, admin_id, content, image_path, likes, comment_count, profile_picture, first_name, last_name, warning_reason.



8.8.4 Deleted Posts (deleted_posts.php)

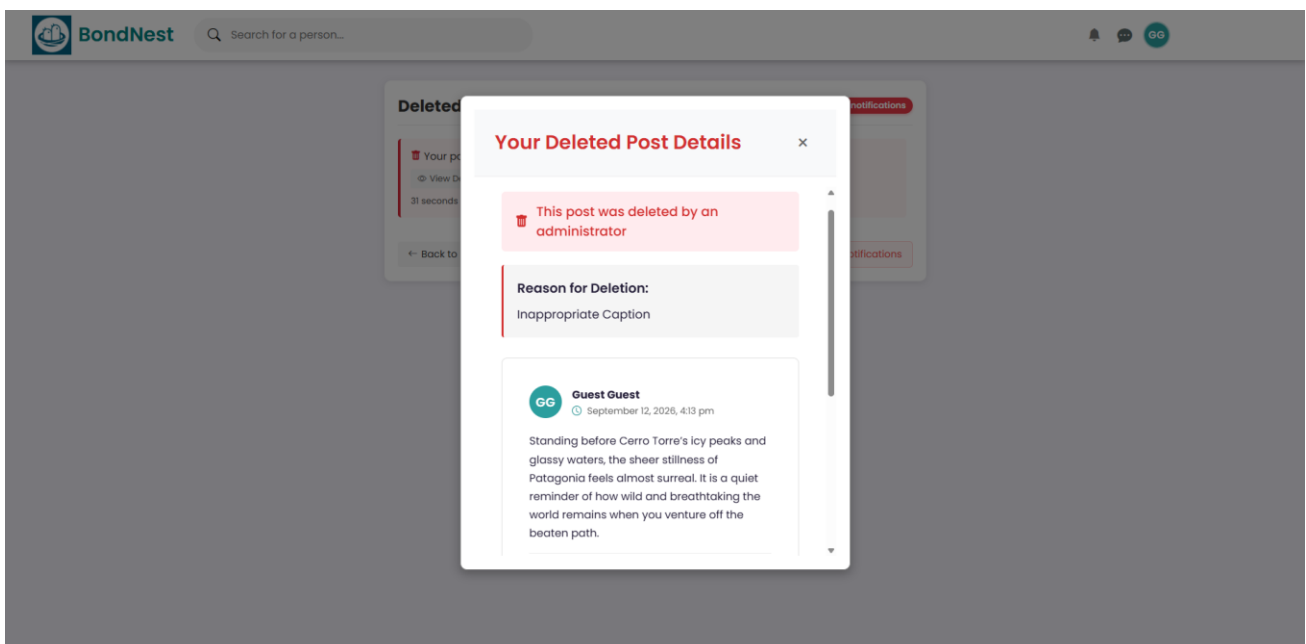
Purpose. Shows notifications when a post is deleted by an admin, with full post snapshot.

Access. Requires login (scoped to own notifications only).

Main UI. List of post_deleted notification cards with full post snapshot, deletion reason, admin name, and timestamps. Mark-read and Clear-All buttons.

Actions. Mark notification as read; clear all read deleted-post notifications.

Server interaction. Creates deleted_posts table if not exists with fields: original_post_id, user_id, admin_id, content, image_path, likes, comment_count, profile_picture, first_name, last_name, deletion_reason, created_at, deleted_at. Queries notifications where type = 'post_deleted'.



8.9 Admin Dashboard (admin.php)

Purpose. Provides administrators with statistics, post moderation, and user management.

Access. Admin only (requires `is_admin = 1` or email matching admin allow-list).

Main UI. Statistics cards at the top (total posts, users, pending, approved, on-hold, warned, deleted). Below: searchable/filterable post table with columns for author, content snippet, status, date, and action buttons (Approve, Hold, Warn, Delete). Each action opens a modal for entering a reason. Navigation links to notification inboxes.

Actions. Approve post (status → approved); hold post (status → on-hold + reason); warn user (snapshot to warnings + notification + reason); delete post (snapshot to `deleted_posts` + notification + reason + row removal). Search/filter posts by status, author, or content.

Server interaction. `get_stats.php` returns dashboard statistics. Post data from `posts` table with author join. `admin_actions.php` / `admin_functions.php` handle moderation actions. `getPostSnapshotForNotification()` embeds a full post snapshot into notifications for detail views. Creates rows in notifications table with types: `post_approved`, `post_on_hold`, `warning`, `post_deleted`.

The screenshot shows the BondNest Admin interface. On the left, a sidebar titled 'Dashboard Stats' displays four cards: 'Total Users' (3, up 0% from last month), 'Total Posts' (1, up 0% from last month), 'Today's Posts' (1, up 0% from yesterday), and 'Today's Actions' (5, up 0% from yesterday). The main content area is titled 'Recent Posts' and includes a search bar and a 'Refresh' button. Below this is a table with columns: ID, USER, CONTENT, IMAGE, CREATED, LIKES, COMMENTS, STATUS, and ACTIONS. A single row is visible for ID 11, user 'Guest Guest', with content 'Standing before Cerro Torre's icy peaks and gl...'. The status is 'Posted'. The actions column contains 'View' and 'Warn' buttons. At the bottom of the table, it says 'Showing 1 to 1 of 1 entries'.

9. Authentication, Authorization, and Account Security

9.1 Sessions and authorization

Successful login/verification stores `user_id` (plus username/admin flag) in the PHP session. Authenticated pages redirect guests to the login page. Admin pages additionally require the admin flag or a configured admin email, else redirect to the homepage. Notification inboxes always scope rows to the viewer.

9.2 Passwords

Hashed with `password_hash()` (`PASSWORD_DEFAULT`) and checked with `password_verify()`. New passwords must be 12–64 characters with upper, lower, digit, and special characters, no spaces, not on a small common-password list, and must not contain the username, email, or names. The same rules run in PHP and in client-side JavaScript. Resets must additionally differ from the old password.

9.3 Email OTP

Six-digit Brevo codes cover registration, password reset, and email change, each with a 10-minute expiry. Without Brevo keys the server logs the code (dev mode) and continues so local testing is unblocked.

9.4 Admin auto-promotion

An account whose email matches BONDNEST_ADMIN_EMAIL (also DIARI_ADMIN_EMAIL / ADMIN_EMAIL, comma-separated, case-insensitive) is promoted to `is_admin = 1` during migration, login, verification, settings, and navbar checks, and routed to the admin dashboard.

10. External Services: Roles and Setup

10.1 Railway

Hosts the PHP web service and PostgreSQL, injects environment variables and PORT, and mounts a volume for uploads. Deploy connects the GitHub repo; the Procfile starts the PHP built-in server against the repo root.

10.2 Brevo

Transactional email for verification, reset, and email-change OTPs via POST `api.brevo.com/v3/smtp/email` with an api-key header, using the dashboard API key and verified sender.

11. Environment Variables and Configuration

Secrets live in Railway Variables, never in git.

Variable	Purpose	How configured
DATABASE_URL	PostgreSQL connection (triggers pgsql mode)	Added automatically when Postgres is linked
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASS	Postgres via parts, or local MySQL name	Operator / defaults (root, empty password locally)
UPLOADS_DIR	Persistent image directory	Railway volume path
BONDNEST_ADMIN_EMAIL (also DIARI_ADMIN_EMAIL, ADMIN_EMAIL)	Admin allow-list for auto-promotion	Railway Variables
BREVO_API_KEY	Brevo HTTP API key	Brevo dashboard
BREVO_SENDER_EMAIL	From address	Brevo verified sender
BREVO_SENDER_NAME	From display name (default BondNest)	Operator
PORT	HTTP listen port	Provided by Railway

Do not commit real values. The screenshot-grade values seen during development (API keys, database URLs) belong in Railway Variables only.

12. Local Development Setup

- Prerequisites: PHP 8.1+ with `pdo_pgsql/pdo_mysql/mbstring/fileinfo/json`, plus MySQL (XAMPP) for local runs.

- Clone the repository and point the web root at the project (or run `php -S 127.0.0.1:8000`).
- Without `DATABASE_URL` the app uses local MySQL (`bondnest_db`, `root`, empty password); `migrate.php` creates/patches tables on boot.
- Leave Brevo keys unset for dev-mode OTP logging, then register and publish a post.
- Note: signup uses a Postgres-only upsert and some LIMIT/OFFSET bindings assume postgres — Railway remains the reference runtime.

13. Railway Deployment

The repo builds via `nixpacks.toml` (`php83` + `pdo_pgsql` + `pdo_mysql`) and starts with Procfile: `web: php -S 0.0.0.0:$PORT -t .` Attach Postgres for `DATABASE_URL`, set the Brevo/admin/upload variables, and mount a volume at `UPLOADS_DIR` so photos survive redeloys.

14. Testing and Current Behavior

Reviewed from the implementation plus operator-described behavior; not an automated suite.

Feature	Expected	Actual / status
Registration + OTP	Pending row + mailed code	Operational (dev-mode log without keys)
Email verification	OTP creates user row	Operational; 10-minute expiry, resend applies
Login	Session on valid password	Operational; generic failure message
Posts / likes / comments	CRUD + toggles + replies	Operational
Messaging	Send/read with badges	Operational
Admin moderation	Approve/hold/warn/delete + notify	Operational; snapshots embedded in new notifications
Notifications	Bell + paginated dropdown + inboxes	Operational
Password policy	Consistent rules everywhere	Operational (PHP + JS)
Uploads	Persist across deploys	Operational when volume mounted

15. Known Limitations and Current System Status

Area	Status
Project completeness	Unfinished second-year project; several flows partial, features missing vs. plan
Dead redirects	<code>logout</code> / <code>profile</code> / <code>message</code> / <code>search_users</code> point to <code>process_login.php</code> , which does not exist (login is <code>index.php</code>)
Debug/test leftovers	<code>test_*.php</code> , <code>debug_*.php</code> , <code>*.log/*.txt</code> , a <code>create_test</code> hook, and a visible Test link ship in the repo
Duplicate files	<code>profile-page backup/modified/bak</code> variants, <code>homepage2.css</code> , <code>*-fix.css</code> , <code>web-images/homepage.php</code>
MySQL parity	Signup upsert is Postgres-only; some bindings assume postgres — Railway is the reference runtime

Migrated admin tool	The Migrate Deleted Posts admin utility was removed pending a rework
Old notifications	Approve/hold items created before snapshots show identity fallback; images only when the post row still exists
Security posture	Student-grade: no CSRF tokens, no rate limiting, no lockouts; generic login errors only

16. Lessons Learned

- One file per page keeps a student project readable, but shared UI (modals, validation) should be extracted before it is copied a third time.
- One-shot admin tools need the same data guarantees as live features — notifications should carry snapshots from day one.
- Environment-driven configuration (DATABASE_URL, admin email, mail keys) keeps one codebase deployable in two places.
- Client clocks cannot be trusted for timestamps; syncing to server time avoids visible jumps.
- Inline onclick handlers cannot reach functions hidden inside DOMContentLoaded closures — expose or delegate.
- Unbreakable strings need CSS-level handling (overflow-wrap + clamping), not just server truncation.

17. Future Improvements

- Fix dead redirects (process_login.php → index.php) and remove debug/test artifacts and duplicate files.
- Restore a reworked deleted-posts migration/backfill admin tool.
- Add CSRF protection, rate limiting, and login lockouts.
- MySQL parity (portable upserts/bindings) or officially drop local-MySQL support.
- Friend/follow graph, richer feed ranking, and real-time updates beyond polling.
- Automated tests and accessibility pass.

18. Project Links

Resource	URL
Live app	https://bondnest.up.railway.app/
GitHub	https://github.com/rproject1324/BondNest

No portfolio or demo-video URLs were supplied; they are omitted rather than invented.

19. Acknowledgements

BondNest is documented here as a single-author, second-year project. The implementation, deployment, and this technical record are the work of Tolentino, Lawrence Dave P.